(c) Cadence Design Systems Inc. Do not distribute.

Jasper® Formal Fundamentals

Course Version 2403

Lab Manual **Revision 1.0**

cadence®

(c) Cadence Design Systems Inc. Do not distribute.

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

(c) Cadence Design Systems Inc. Do not distribute.

Introduction

These lab exercises are designed to make you more familiar with using Formal Analysis effectively in an Assertion-Based Verification (ABV) methodology using the Jasper® tool. They are organized in a way that takes you step by step through what to do first, and then slowly removes the guidance as you become more familiar with the tools and methodology.

This course is suitable for intermediate users and newcomers to Formal Analysis (FA) and Jasper. Formal Analysis is a completely different paradigm to older and more widely adopted methods of verification like simulation.

As such the fundamental objectives, capabilities, limitations, setup and initialization requirements, and analysis and interpretation of results need to be understood to make effective use of FA tools.

Formal is capable of much more than simulation alone can deliver but optimization techniques often need to be applied in order to get conclusive results within realistic timescales.

These techniques are extremely difficult to discover or pick up “as one goes along” without training or expert assistance.

Anyone can run an FA tool, but experience is needed to get conclusive results in a realistic timeframe which are completely understood. This course is intended as a shortcut to gaining that experience.

Overview of Labs

The labs are provided in SystemVerilog HDL and SystemVerilog Assertions.

The Language Reference Manual (LRM) for both of these is specified in IEEE 1800-2017. The previous version was IEEE 1800-2012.

The first lab is intended to familiarize you with the tools that you are using and the basic steps involved in verifying simple SVA assertions with Jasper.

Much of the required setup has already been done for you for the first lab. We will discuss in more detail in later lectures and labs all of the required setup.

The first lab is intended to introduce you to how to run Jasper and how the results are presented to you before we discuss the tool and methodology in more detail.

The goal is not to necessarily complete all the exercises during the course but to learn from each exercise at your own pace.

(c) Cadence Design Systems Inc. Do not distribute.

Lab Instructions

1. Software Requirements:

   a. Required Jasper Version: Jasper_2024.03 FCS +

   b. Cadence VIPCAT 11.30.095

2. Untar lab Database file using the following command:

   ```
   tar -xzvf JFF_2403.tar.gz
   ```

3. Go to JFF_2403 and open class_setup file in an editor.
   If you are not using a Cadence-provided tool environment, the env variables PATH, LM_LICENCE_FILE, ABVIP_INST_PATH, and CDN_VIP_ROOT will be unique to your company's environment. You must find out what these settings should be inside your own company. Often, these are set using Linux "module load" commands. You may care to edit the class_setup file with the correct value for those env vars or rely upon your company's mechanism, for example, using "module load".

4. Once the class setup file is edited with your tool environment paths, change the directory and source the class setup file using the following command.

   ```
   cd JFF_2403
   source class_setup
   ```


© Cadence Design Systems, Inc

(c) Cadence Design Systems Inc. Do not distribute.

Table of Contents

Jasper Formal Fundamentals

Lab 1 **Simple Assertions to Verify FSM 7**
Objective 7
Scenario 7
Connection Definitions 7
Output Decode 9
Simple Boolean Assertions 9
Testing the Boolean Assertions 10

Lab 2 **Benefits of Formal Analysis 11**
Objective 11
Introduction to the Frame Module Specification 11
Action 15

Lab 3 **Developing Blackbox Protocol Properties 18**
Objective 18
Transfer Engine Description 18
Expressing Required Blackbox Protocol Properties 21
Blackbox Protocol Property Development in SVA 22
Expressing Required Cover Statements 23
Cover Statement Development in SVA 24

Lab 4 **Basic Complexity Handling 27**
Objective 27
PART 1: Reducing array/flop count 27
PART 2: Reducing gate count 31
PART 3: Reducing assumptions 32

Lab 5 **Complexity Reduction Techniques 34**
Objective 34
Initial Value Abstraction (IVA) 34
Scenario 34
Conditions 34
Connection Definitions 34

© Cadence Design Systems, Inc.

(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

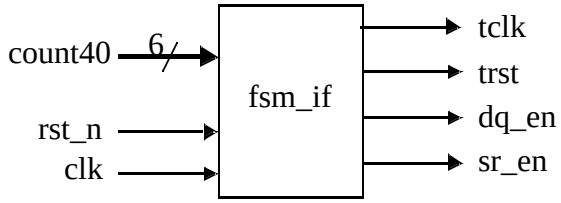
Lab 1 Simple Assertions to Verify FSM

Objective

To understand the distinction between same-cycle and next-cycle implications in SystemVerilog assertions and to gain experience of using Jasper® to verify and debug RTL designs

Scenario

The design is a simple FSM-based interface controller:



Connection Definitions

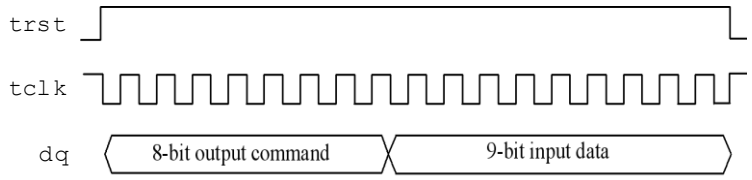
Port	Direction	Type	Description
count40	input	count value 0 to 39	Counter for serial frame
rst_n	input	single bit	master active low reset
clk	input	single bit	master clock
tclk	output	single bit	serial interface clock
trst	output	single bit	serial interface frame
dq_en	output	single bit	tri-state enable
sr_en	output	single bit	shift register enable

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

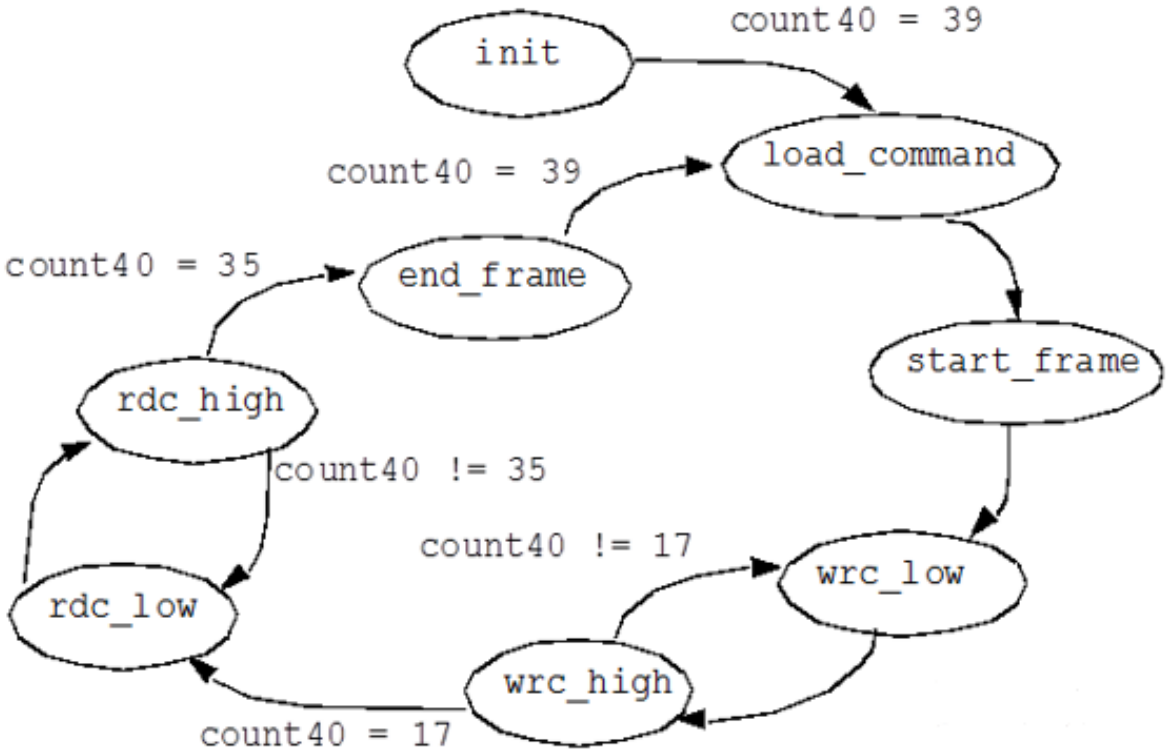
The FSM controls a 3-wire bidirectional serial interface to an external device. Each frame of data over the interface consists of an 8 bit serial output command followed by a 9 bit serial input response from the device.

The transaction is clocked by *tclk* and framed by logic 1 on *trst*. *trst* and *tclk* are driven directly from the FSM, whereas *dq* is driven by external logic, controlled by the FSM outputs *dq_en* and *sr_en*.



State Diagram

The states and state transitions for the FSM are as shown here.



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

HDL FSM State Encoding	
State Name	**Value of `reg state[2:0]`**
init	0
load_command	1
start_frame	2
wrc_low	3
wrc_high	4
end_frame	5
rdc_low	6
rdc_high	7

Output Decode

The required outputs for each state are shown here.

State	Outputs {ops}			
	`tclk`	`trst`	`dq_en`	`sr_en`
init	1	0	0	0
load_command	1	0	0	0
start_frame	1	1	0	0
wrc_low	0	1	1	1
wrc_high	1	1	1	0
rdc_low	0	1	0	1
rdc_high	1	1	0	0
end_frame	1	0	0	0

Simple Boolean Assertions

5. Change your working directory to `<Labs Install Dir>/lab1_fsm/jg`, for example: `/home/user1/JFF_XXXX/lab1_fsm`.

6. A model for the FSM is provided in the `lab1_fsm/fsm_if.v` file.

7. Open the `fsm_if.v` file and familiarize yourself with the design. Do not modify any of the FSM source code at this time.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

8. Write named properties to check the outputs for each state in the FSM using the same-cycle implication operator (|−>) and assert them.
   - The 4 outputs of the FSM have been collected into a vector called *ops* for convenience. This is how each bit of the vector relates to each output:

     ```
     ops = {tclk, trst, dq_en, sr_en};
     ```

   - The design uses a SystemVerilog *localparam* type to define the state variable, so we use the state name when checking the state variable, such as:

     ```
     (state == start_frame)
     ```

   - The assertions are very similar to each other. Use copy and paste when creating these and the following assertions!

9. Write named properties to check the conditional state transitions of the FSM using the next-cycle implication operator (|=>) and assert them.

10. Compile the FSM and correct any bugs in the assertions.

Testing the Boolean Assertions

1. A formal testbench for the FSM is supplied in the `lab1_fsm/fsm_if_test.v` file in order to correctly constrain the design count input to act as a counter would be expected to in the real design.

2. Compile and elaborate by running the *./RUNME.sh* script. Run the proofs using the Tcl command `prove -all` and check your assertion results.

3. There are two deliberate errors in the FSM implementation compared to the design specification. Your assertions should detect and report these errors!

4. Fix the errors in the FSM design. Rerun the tool to check the errors have been corrected.



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 2 Benefits of Formal Analysis

Objective

Show an area of great Return On Investment (`ROI`) with Formal Analysis Scenario

1. We will be writing properties to check that a specification has been met and then the RTL will implement the specification. We will check that the implementation meets the specifications (the properties) of Jasper®. No testbench or simulator will be used. Formal works extremely well if used while designing blocks, so we will see how we can produce RTL that meets a specification without the need to create a testbench or think about the stimulus required to exercise all of the corner cases. This gives us more confidence that the functionality of the block is correct when integrating the block into larger-scale designs.

2. The framework of the design is provided, but you have to write properties to check the functionality of the design and the RTL to implement that functionality.

3. People often write properties before writing any RTL. If you can write meaningful properties about the functional requirements defined in the specification, then this means that you fully understand the specification and are therefore more likely to be successful in implementing the RTL to meet the requirements of the specification. In addition, the act of writing properties can find errors or ambiguities in the specification itself.

Introduction to the Frame Module Specification

1. The code to be written is for a module called `frame` that receives bad (corrupt) packets/frames and transmits good packets/frames.

2. The purpose of the `frame` module is to transmit packets with proper framing. It is able to receive packets with framing errors (corrupt packets), but it cannot transmit those errors. Instead, it needs to transmit corrected packets. To do this, the module will ignore all input conditions that do not adhere to the framing protocol.

3. Because the inputs can be erroneous and violate protocol, it helps to think of them as being purely random. Therefore, we do not need to specify any constraints (assumptions) on the design inputs.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Framing Protocol

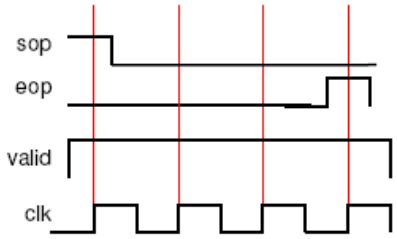
1. The framing for a given packet is delimited by two signals: a “start of packet” (`sop`) signal and an “end of packet” (`eop`) signal. At the beginning of a packet, `sop` is asserted for one cycle.

2. The last cell in a packet is indicated by the assertion of `eop` for one cycle. A packet can be only one cycle long, i.e., `sop` and `eop` are asserted at the same time. There is also a `valid` signal that, when asserted, indicates `sop` and `eop` are valid. An `sop` framing error occurs if there are two `sop's` without an `eop` in between. An `eop` framing error occurs if there is an `eop` without having a corresponding `sop`.

3. We define the protocol such that a `valid` signal is allowed to be deasserted in the middle of a packet transmission, i.e., `valid` can be deasserted at any time for any number of cycles, and we ignore what `sop` and `eop` inputs to the `frame` module are doing if `valid` is deasserted. This means that the `valid` signal can pass straight through the `frame` module. The `frame` module should never output a corrected `sop` or `eop` without a `valid` signal being asserted. The `frame` module also has an active low reset. When reset is active i.e., `'0'`, the `sop` and `eop` signals out of the `frame` module should be deasserted, i.e., `'0'`.

4. `Figure 1` below depicts a representation of a good packet.

FIG1 - Good Packet

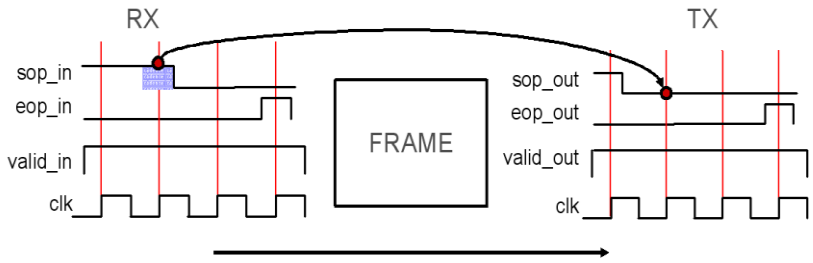


© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

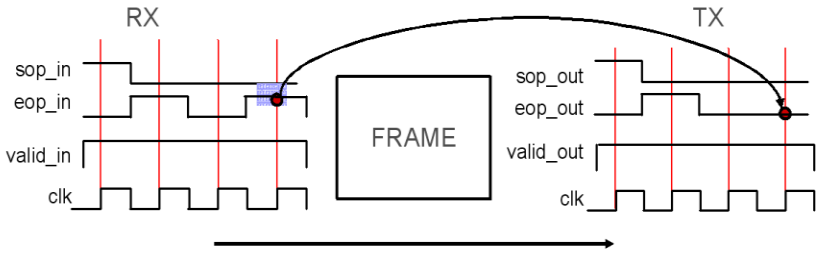
5. `Figure 2` illustrates an example of an `sop` framing error on the receive side and how the output looks after correcting the error. The `sop_in` signal starts two packets. The second packet is started before the first packet is completed. The transmit (TX) side shows how this error is corrected.

FIG2 - Corrected SOP



6. `Figure 3` illustrates an example of an `eop` framing error on the receive side and how the output looks after correcting the error. The second `eop_in` signal occurs without a corresponding `sop_in` – hence is masked out. The transmit (TX) side shows how this error is corrected.

FIG3 - Corrected EOP



7. `Figure 4` shows several examples of good and bad packets.

8. A good packet of length 2 is sent during cycle 1 and 2.

9. This is followed by a good packet of length 1 in cycle 3.

10. In cycle 4, an error occurs when `eop` is asserted but no corresponding `sop` is present.

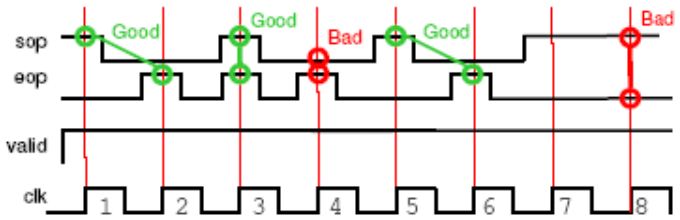
11. In cycles 5 and 6, the `sop` and `eop` behave properly, forming another two-cycle packet.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

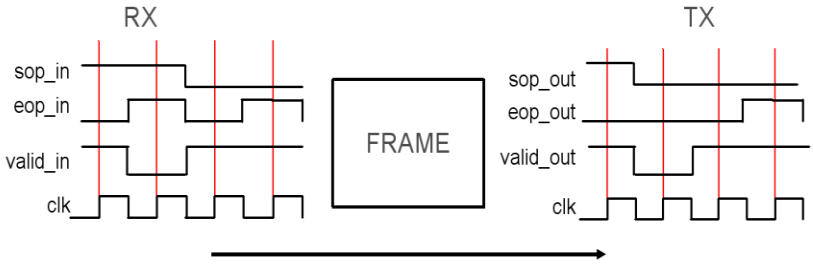
12. Finally, in cycle 7, a new packet starts when `sop` is asserted, but in cycle 8, `sop` is asserted again before the previous packet is completed.

FIG4 - Good and bad packets



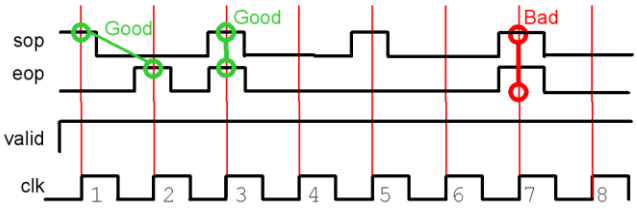
13. `Figure 5` shows the effect of the `valid_in` signal. When `vaild_in` is deasserted, then the `frame` module ignores the value of `sop_in` and `eop_in`, hence `sop_out` and `eop_out` are not asserted while `valid_out` is deasserted. Remember that `valid_in` is connected straight through to `valid_out`.

FIG5 - Effect of input qualification with `valid_in`



14. Figure 6 shows an example of two good frames followed by a bad frame. The bad frame starts on cycle 5. On cycle 7, an `eop` occurs to terminate the frame, but on the same cycle `sop` is asserted, hence we are starting and ending a frame at the same clock, which means we have overlapping frames. This is not allowed in the protocol.

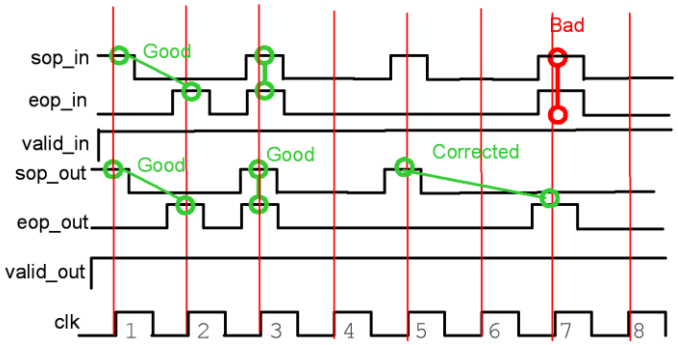
FIG6 - Overlapping frames



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

FIG7 - Alignment of inputs and outputs



Note: There is no delay between a "good" `SOP_IN` and `SOP_OUT`. They must occur at the same time. The same applies to `EOP_IN` and `EOP_OUT`.

Action

Writing the Properties

1. Change your working directory to `<Labs Install Dir>/lab2_DLFA/jg`, for example: `/home/user1/JFF_XXXX/lab2_DLFA/jg`.

2. Write your SVA properties as embedded properties in file `../rtl/frame.v`. Don't edit any HDL code at the moment. We will write the properties first in order that we understand the design functional requirements.

3. It is completely up to you how you describe properties to verify that the functional requirements have been met, but here are some suggestions on what you could write your properties about:

   No `eop_out` without a corresponding `sop_out`.

   No `eop_out` or `sop_out` without a `valid_out`.

   Never two `sop_out` without corresponding `eop_out`.

Writing the RTL

1. If you have written the properties, then you must have understood the specification.

2. Write RTL code to meet the functional requirements of the specification in file `../rtl/frame.v`.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

3. Again, it is completely up to you how you implement the RTL, but some hints are given below which may be of assistance.

4. Because the inputs are allowed to be erroneous and violate protocol, it helps to think of them as being purely random. Based on this insight, one approach would be to mask out `sop_in` such that only “good” `sop's` are transmitted to the output. To do this, consider creating a new mask signal that can be used to mask out bad `sop's`. The key then is to determine how to generate the mask from current inputs to the system and any state that you create in the design (i.e., you need to "remember" some condition). The same would apply to masking out "bad" `eop's`.

Running the Proof

1. The scripts have already been prepared for you. If you are interested, you may want to inspect the Jasper Tcl file `jg_frame.tcl`. We do not constrain the behavior of the `sop_in`, `eop_in` and `valid_in` signals because we have already stated that we can view their behavior as being random.

2. Change your working directory to `<Labs Install Dir>/lab2_DLFA/jg`, for example: `/home/user1/JFF_XXXX/lab2_DLFA/jg`

3. To launch Jasper from the OS command line, enter the following command:

   ```
   jg jg_frame.tcl
   ```

4. Use Jasper to debug any errors until all of your properties pass. You can use the shell script `./CLEAN` to ensure that previous runs are deleted. See your instructor if your properties are failing but you don’t understand the CEX.

5. In the Jasper Tcl console, enter `exit` to close the application.

The Acid Test

1. We have prepared a verification component file which defines properties which check for the correct behaviour of your frame module. This is in file `frame_vcomp.v`. We have also prepared a script which called `jg_frame_acid.tcl` that will use the SVA statements in the file `frame_vcomp.v` as well as the SVA statements that you have inserted in file `frame.v`. This allows a direct comparison of your properties and the solutions.

2. Execute the script `./ACID_RUN.sh`.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

3. See if your RTL implementation meets our definition of the correct properties that the frame module should have. Debug any failures.

4. Take a look at how the `frame_vcomp.v` properties have been implemented and read the associated comments. Notice how the use of auxiliary code helps us to efficient and write simple properties where to don't need to be concerned about the intricacies of liveness properties.

5. Note that one property that is expected to fail and take time to understand the reasons why.

Conclusions

1. It can be extremely useful to define properties before RTL as this tends to debug ambiguities in the specification and leads to a greater understanding of the requirements before the RTL implementation is started.

2. Formal Analysis is extremely effective if used while a block is being designed as the design knowledge required to write properties is still fresh in the designer's mind.

3. It is difficult to create a testbench that tests all corner cases. With formal, you don't need to consider this as all states will be explored and you didn't need to create a testbench.

Extra Credit

1. If you have the time, then look at the scripts `./ACID_RUN_SOL*` and observe the difference in the files which get processed and compare the results of each script.



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

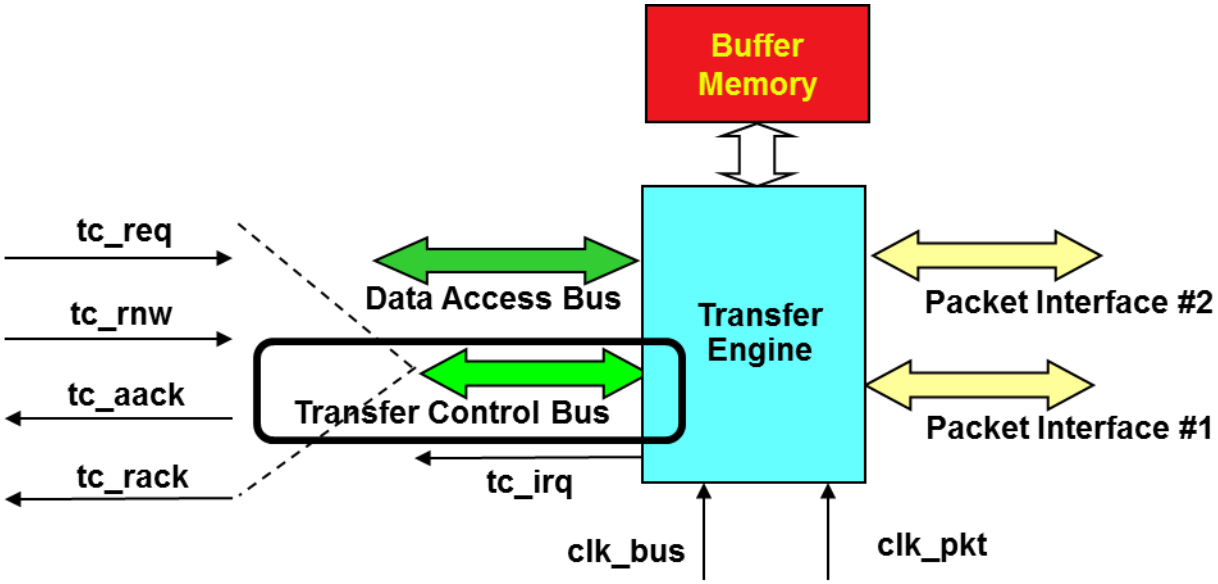
Lab 3 Developing Blackbox Protocol Properties

Objective

The objective of this lab is to create and verify Blackbox Protocol Properties for one of the interfaces of the Transfer Engine described in the lectures. This will be done following the system of methods stated in the lectures.

Transfer Engine Description

The purpose of the `Transfer Engine` is to provide automated data transfers to and from various interfaces. The figure below shows a diagram illustrating the main interfaces of the design.



Transfer Engine Interfaces

The interface we will be working on in this lab is the Transfer Control Bus. This interface is a fairly straightforward bus with the added complexity of pipelined operation. The interface supports up to 4 in-flight transactions which can be any combination of reads and writes. Pipelining enables higher performance in the face of longer latency operations that the Transfer Control Bus interface supports. Addresses refer to 4 byte locations inside the Transfer Engine memory map.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Transfer Control Bus I/O Description

tc_req	1	In	This is the request line. This signal is active high and indicates that either a read or a write request is on the bus. The maximum number of outstanding requests cannot be more than four. These outstanding requests can be any mix of read and write requests. Once the `tc_req` request line goes high, it must stay high until acknowledge by `tc_aack`.
tc_rnw	1	In	This is the read/write modifier. When high, it implies that the requested transaction is a read transaction. When low, it implies that the requested transaction is a write transaction. `tc_rnw` should be driven when `tc_req` is asserted and should stay stable up to and including the cycle that `tc_aack` is asserted.
tc_aack	1	Out	This is the address acknowledge signal. When this signal goes high, it indicates that a transaction has been acknowledged and the write data (if applicable) and address have been latched. `tc_aack` can be asserted speculatively, if it can be guaranteed that a new transaction will be accepted. If there are four pending transactions, `tc_aack` should no longer be asserted until at least one of the pending transactions has been completed.
tc_addr	5	In	This is the address for the currently requested bus transaction. This address refers to a 32-bit location inside the Transfer Engine. `tc_addr` should be driven when `tc_req` is being asserted and should stay stable up to and including the cycle that `tc_aack` is asserted.
tc_wack	1	Out	This is the write data acknowledge. This signal will go high for one cycle when a write transaction has completed. The purpose of this signal is simply to inform the associated master that an internal transaction queue entry has been freed. The slave must always guarantee that the write data for a particular address that has been accepted (but not necessarily completed) is accurately reflected in a subsequent read to the same address.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

tc_wdata	32	In	This is the write data for the Transfer Control Bus. This data will be written as appropriate to the internal registers. `tc_wdata` should be driven when `tc_req` is being asserted and should stay stable up to and including the cycle that `tc_aack` is asserted.
tc_rack	1	Out	This is the read data acknowledge. This signal will go high when the read data bus has valid read data. Read data is valid only for a single clock cycle. There is a one-to-one correspondence between read requests and `tc_rack` so that if `tc_rack` is asserted, it implies that sometime in the past, there was a read request. If `tc_rack` is asserted during two back-to-back clock cycles, it implies that there were two corresponding read requests (possibly to the same address).
tc_rdata	32	Out	This is the read data. Data is to be valid on this bus when `tc_rack` is asserted.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Part 1: Expressing Required Blackbox Protocol Properties

First, we will develop a set of properties for the transfer control bus. Initially, we will express these properties in natural language as opposed to an assertion language like SVA. The objective is to recognize what we want to achieve before coding using an assertion language.

Recall from the lectures that the protocol behaviors could be all of:

Behavior	Description
Condition	Requirement that restricts the values of signals, or imposes a relationship among them
Stability	Requirement that a signal must be stable during a given period
Response	Requirement that one or more signals must change in response to an event, or that a relationship must hold following an event
Stimulus	Requirement that a given response must only occur following the appropriate stimulus
Sequence	Requirement regarding sequences of conditions or events occurring in the design

However, this does not necessarily mean that you could write a property for each of the behaviors for this particular bus protocol.

1. Use the Worksheet 1 at the end of this lab description to express the properties required in natural language. Think in terms of the protocol property patterns and behaviors. In addition to using the interface signals for your properties, you could also make use of the following auxiliary code registers which hold an integer value:

Register Name	Purpose
pending_rd_cnt_ff	Number of pending read transactions, i.e., number of requested read transactions that have not yet had a corresponding `tc_rack`
pending_wr_cnt_ff	Number of pending write transactions, i.e., number of requested write transactions that have not yet had a corresponding `tc_wack`. Note that as `tc_aack` can be speculatively held high and no data needs to be fetched, a write transaction can be acknowledged immediately without `pending_wr_cnt_ff` being incremented
pending_cnt	pending_rd_cnt_ff + pending_wr_cnt_ff

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

2. Identify the properties as either assertions or constraints and apply the corresponding verification directive.

Blackbox Protocol Property Development in SVA

We will now write properties in an assertion-based language to meet a description in natural language:

1. Change your working directory to `<Labs Install Dir>/lab3_BB_IF/part1/Jaspergold`: for example `/home/user1/JFF_XXXX/lab3_BB_IF/part1/Jaspergold`

2. Open the file `./vcomp_tcbus.v` in a text editor.

3. Go to the commented section titled `Protocol Properties`

4. Do the comments express properties that match your definitions from your worksheet? Before continuing, make sure you understand the requirements of the properties and how they relate to the properties that you identified on the worksheet. Discuss with your instructor if you wish.

5. Translate each natural language comment into the associated SVA representation.

6. Determine which properties should be assertions and which should be assumptions.

7. Run Jasper® by entering at UNIX command prompt:

   ```
   jg tcbus.tcl
   ```

8. Fix any compilation/elaboration errors with your properties.

9. Run the proof on your assertions by entering the command `prove -all` on the Jasper command line.

10. If your assertions and constraints are correct, then all trigger checks and assertions should pass except for one. The failing assertion should be the one which checks `tc_aack` should not be asserted if the pipeline is full.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Part 2: Expressing Required Cover Statements

We stated in the lecture that cover statements are very important ones to add to your verification environment. This is because they can be used to aid in the following:

1. Used to identify and debug over-constrained environments
2. Verification of blackbox protocol properties
3. Verification completeness

We will write cover statements to serve as verification completeness for the properties you developed previously.

Use the Worksheet 2 at the end of this lab description to express the cover statements required in natural language.

Remember that we can still make use of the auxiliary code registers that you used previously.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Cover Statement Development in SVA

We will now write cover statements in an assertion-based language to meet a description in natural language:

1. Change your working directory to `<Labs Install Dir>/lab3_BB_IF/part2/Jaspergold`, for example: `/home/user1/JFF_XXXX/lab3_BB_IF/part2/Jaspergold`.

2. Open the file `./vcomp_tcbus.v` in a text editor.

3. Go to the commented section titled `Cover Statements`

4. Do the comments express properties that match your definitions from your worksheet? Before continuing make sure you understand the requirements of the properties and how they relate to the properties that you identified on the worksheet. Discuss with your instructor if you wish.

5. Translate each natural language comment into the associated SVA representation.

6. Run Jasper by entering at UNIX command prompt:

   ```
   jg tcbus.tcl
   ```

7. Fix any compilation/elaboration errors with your cover statements.

8. Run the proof on your covers by entering the command `prove -all` on the Jasper command line.

   All of your cover statements should pass.



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Worksheet 1

Stimulus Properties
English : SVA :
English : SVA :
English : SVA :
English : SVA :
Stability Properties
English : SVA :
English : SVA :
English : SVA :
English : SVA :
Response Properties
English : SVA
English : SVA :
English : SVA :
English : SVA :
Conditional Properties
English : SVA
English : SVA :
English : SVA
Sequence Properties
English : SVA
English : SVA :

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Worksheet 2

Cover Statements
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :
English : SVA :

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 4 Basic Complexity Handling

Objective

This lab has the intention to reduce the complexity of a liveness assertion. The assertion is proven out of the box, but we would like to demonstrate how the complexity can be reduced by under-constraining the design with different methods. Note that the results you observe depend heavily on your Farm infrastructure, the machine you are on (if run locally) and engine algorithms. However, an obvious trend of performance improvement should be expected and observed.
You will need the VIP Catalogue (VIPCAT) installed to do this lab. It should already be done for you and in addition the following environmental variables should already be set. Example environment variables are shown below:

```
setenv CDN_VIP_ROOT "/grid/avs/install/vipcat/11.3/latest"
setenv ABVIP_INST_DIR  "${CDN_VIP_ROOT}/tools/abvip"
```

PART 1: Reducing array/flop count

1. Change your working directory to `<Labs Install Dir>/lab4_complexity`, for example: `/home/user1/JFF_XXXX/lab4_complexity`

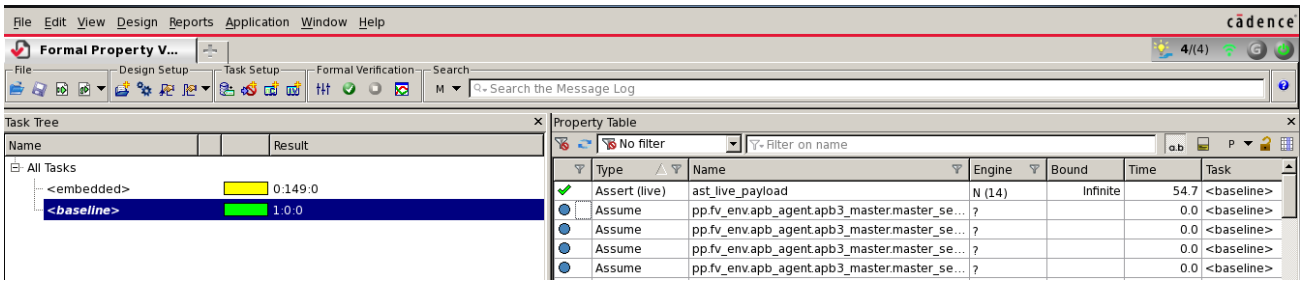
2. Invoke Jasper® with the Linux command `$> jg tcl/run.tcl`.

3. Throughout the lab we are focusing on the liveness assertion `ast_live_payload`. The `run.tcl` script proves that assertion. Use the column filters to exclude unprocessed properties if you wish. It may take several minutes before that single assertion is proven. To recall how long the proof takes, we can look back at this proof in the <baseline> task later.

4. Note that we use orchestration and a different engine might prove the assertion in your case in a slightly different time.

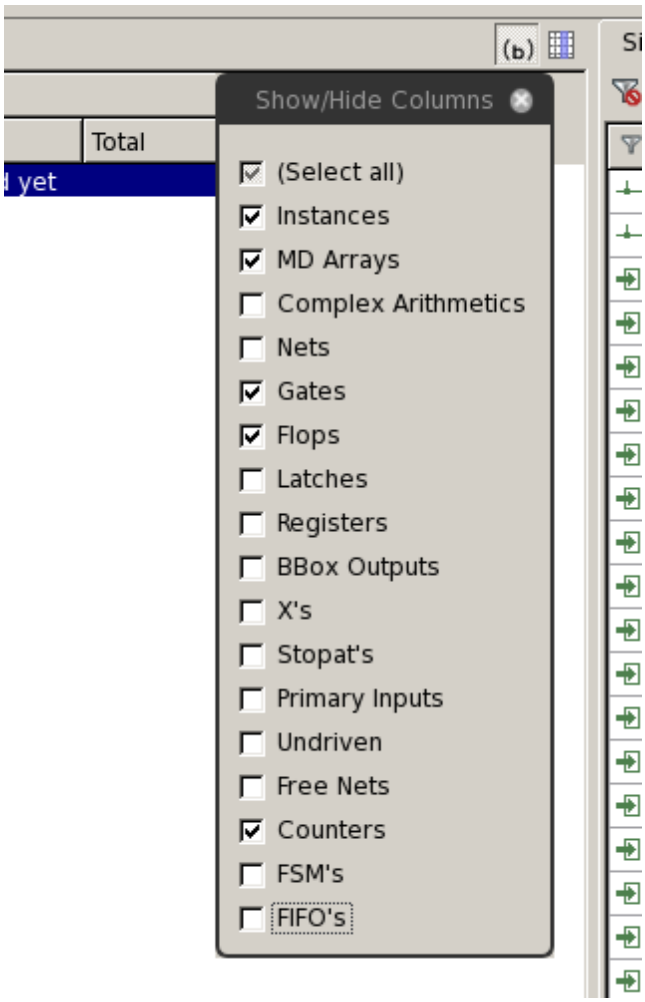
© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.



5. Let’s have a look with the complexity manager on the COI of the property. Open the complexity manager by right-clicking on the property and select “Complexity Manager”. It is also possible to open the GUI via Tcl using,
```
complexity_manager -property
{<baseline>::ast_live_payload}
```

6. On the left-hand side of the complexity manager area, change the columns that are displayed to: Instances, MD Arrays, Gates, Flops, and Counters.

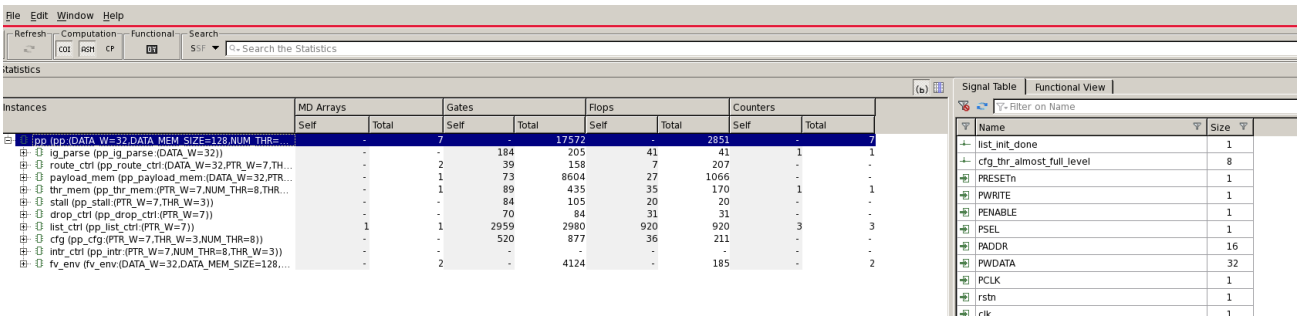


© 2024 Cadence Design Systems, Inc. All rights reserved.

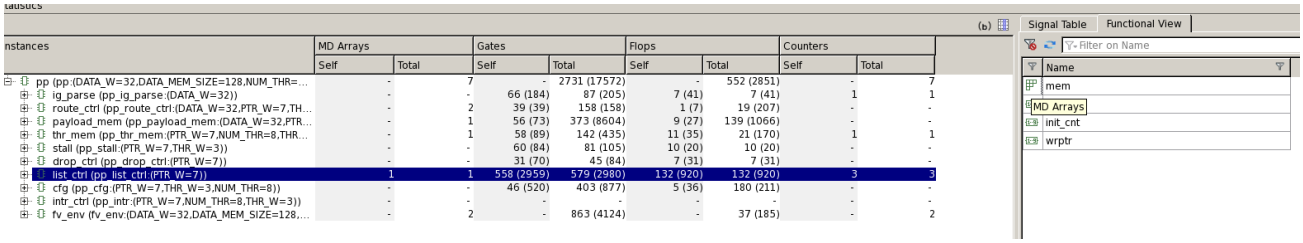
(c) Cadence Design Systems Inc. Do not distribute.

7. Note that the statistics on counters are not computed yet. Select the button for the functional statistics: 

and select counters. Click the “Refresh” button to update the statistics and expand the hierarchy in the Instances column.



8. Let’s first focus on big arrays. Usually, we black-box big arrays or memories during elaboration. Sometimes we need to keep them or build an easier model to avoid false negatives. In this example, we are focusing on one single liveness assertion, and it’s possible to do a more selective black-boxing. In the MD Arrays column, you see that in instance list_ctrl there is one. Select the instance, and on the right-hand side select the functional view. When hovering your mouse on the icon to the left of the `mem` signal, you see that this is a MD array.



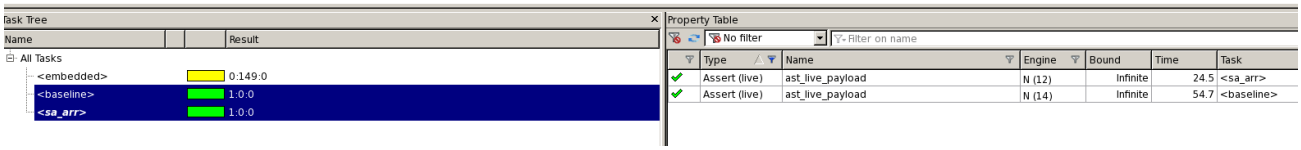
9. Let’s try to abstract that array by putting a `stopat` on it. Because we do not know if this will lead to a false CEX for this assertion and others in the original task, we create a new task and copy the environment and apply the `stopat` in the new task. We run a proof for sanity checking :

```
% task -create <sa_arr> -source_task <baseline> -copy_all -set
% stopat list_ctrl.mem
% prove -task .
```

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

10. The property should be proven, but in this task the engines should be able to prove it faster! Take a note of how long the proof takes this time and compare that to the previous run in the `<baseline>` task.

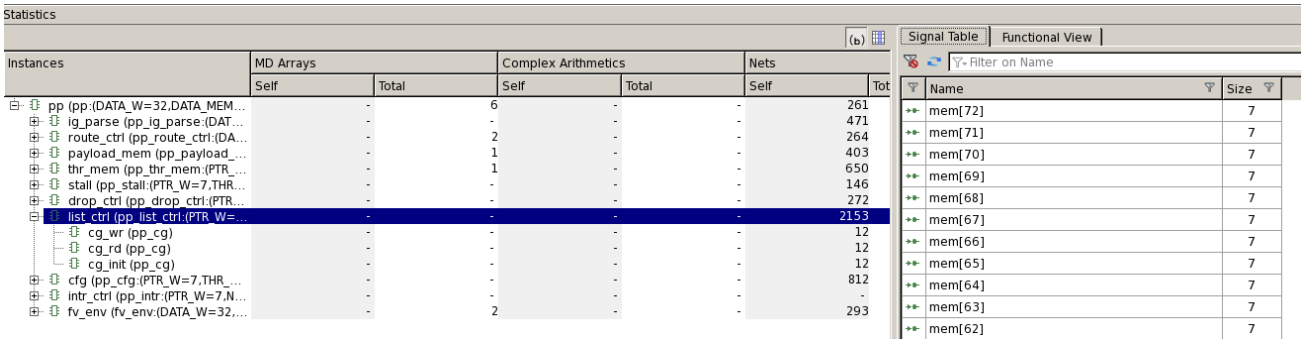


© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

PART 2: Reducing gate count

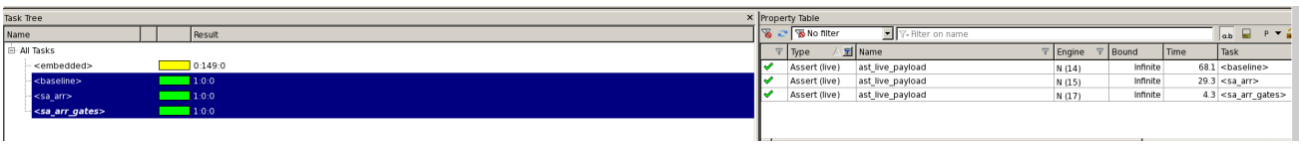
1. Not only flops and large arrays are contributing towards complexity. Instances with lot of combinatorial logic can cause a complexity bottleneck. Let's open the complexity manager on the property again, but now in the `sa_arr` task. Note: if the CM GUI window is greyed out, then you might need to press the refresh button to update the information. First, let's check the `list_ctrl.mem` array again. In the MD Arrays column on the left-hand side the MD array is gone. In the signal table, the MD vectors are still displayed but the icon indicating that there is a `stopat` on it.



2. It's indicating that the signal has a `stopat` on it. In the complexity manager, you can see that instance `list_ctrl` and `payload_mem` have the highest gate count. Instance `fv_env` is the formal environment, so we will not touch this for the moment. Let's put a `stopat` on the instances and run a proof for sanity checking.

```
% task -create <sa_arr_gates> -source_task <sa_arr> -copy_all -set
% stopat payload_mem
% stopat list_ctrl
% prove -task .
```

3. The property should be proven, again with a faster proof time.



© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

PART 3: Reducing assumptions

1. Assumptions can drag a lot of logic and complexity into a proof problem. We use the Complexity Manager to check that on the example. Note the 3 buttons in the Computation group.



2. When selecting ASM, the COI of **all** assumptions in the task are included in the complexity analysis. Use the ASM button to change between the two different nodes – a refresh might be necessary. Notice the difference in the number for gates and flops.

3. Let's create another task and run `get_needed_assumptions` to find a minimal set of assumptions to converge on the property and avoiding false negatives.

```
% task -create <sa_arr_gates_gna> /
    -source_task <sa_arr_gates> -copy_all -set
% get_needed_assumptions /
    -property ast_live_payload /
    -proofgrid_mode local /
    -attempts_time_limit 15s -max_parallel_jobs 4 /
    -status {undetermined proven} -include_liveness_assumes
```

4. Note the options we use for `get_needed_assumptions`. We are parallelizing with 4 jobs and run locally to increase the speed. We have selected an attempts time limit of 15s. For other examples this might be too small, and a higher time limit is needed. We need to include liveness assumptions because we are investigating a liveness assertion. Note that we are using the status undetermined and proven – we are looking for the subset of assumptions that we need to avoid false negatives.

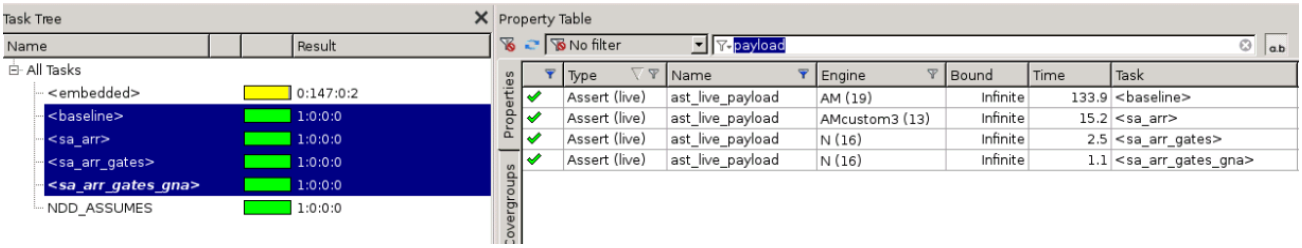
5. `get_needed_assumptions` should return 9 assumptions that are needed in order to avoid a false CEX. We are enabling those assumptions and disabling the others in our current task.

```
% set ndd_list [get_property_list -include {type assume disabled 0} /
   -task NDD_ASSUMES -no_task_prefix -exclude_env]
% task -set <sa_arr_gates_gna>
% assume -disable *
% foreach a $ndd_list { assume -enable $a }
% prove -task .
```


© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

6. Look at the results in all four tasks highlighted below and compare the runtime. Note that the times you achieve will not be exactly the same, due to different machines, server farm latencies, orchestration choosing different engines and so on . However, the approximate times and the trends observed should be the same.



7. Reminder: Complexity manager can show you structures that are in the COI of a property. It lists complexity inducing design elements that may lead to performance problems. It does not show, where engines spend their time or which logic is potential irrelevant for the engines. The formal profiler can help the user to focus the complexity reduction by listing signals/instances that are difficult for the engines.

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 5 Complexity Reduction Techniques

Objective

The objective of this lab is to try the basic underconstraint method and observe the results with simple examples.

Initial Value Abstraction (IVA)

Implement Initial Value Abstraction (IVA) on counter to jump on the critical value and help assert to find the inserted bug faster.

Scenario

The design is a simple counter that counts whenever in_cntr is HIGH and reset to 0 when reset_cntr is HIGH.

Conditions

These two conditions must be satisfied for the implemented counter.

Counter should increment by one whenever in_cntr is HIGH.

Counter should not overflow.

Connection Definitions

Parameter	Description
depth	depth of the counter(how much it can count)

Port	Direction	Type	Description
rst_n	input	single bit	master active low reset
clk	input	single bit	master clock
reset_cntr	input	single bit	Counter reset
in_cntr	input	single bit	Counter count increment
current_count	output	[$clog2(depth)-1:0]	Current count value

© 2024 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

out_cntr_full	output	single bit	High when the counter is full

Implementation

As mentioned, there are inserted bugs, the main task is to use IVA to find bugs faster.

The abstraction is implemented by abstracting the **init value** of the counter. It will be possible to let the counter start from the critical value and start counting from that value.

Two assertions are written for two given conditions.

IVA Overview

At a conceptual level, the steps needed to apply the IVA described above are:

- Abstract the init value of the counter.
- It will start from a critical value/warm state.

Creating IVA in Tcl

1. Go to the test directory for this part of the lab:
   `<Labs_Install_dir>/LAB5_CRT/IVA/`
2. Run tcl script *`chk_iva_cntr.tcl`* in Jasper® (or run RUNME.sh in the command line) and Observe the proof being slow. (It will likely timeout at 5 minutes..)
3. Open Tcl script *`chk_iva_cntr.tcl`* to edit.
4. Look at the “if {$iva}” block. You will see numerous lines there that you are expected to fill in. Try to add the appropriate *abstract* command as instructed in the comments. (If you get stuck, you can look in the .solutions dir…)
5. When confident in your additions, change the “set iva 0” to “set iva 1”, and run again.
6. You should see a CEX appear relatively quickly.
   a. The CEX length is likely lower than the previous proof bound. Is this OK? Why?
   b. Look at the CEX, and check the values of cntr. Did Jasper take advantage of your Initial Value Abstraction to start at a warm state?
7. Observe the result and compare both the result.

End of Lab

© 2024 Cadence Design Systems, Inc. All rights reserved.